Diseño, análisis y comparación de algoritmos para la Subsecuencia Común Más Larga

Nombre y apellidos del estudiante
Actividad 1 · Análisis de Algoritmos

Fecha de entrega: 10 de noviembre de 2026

Resumen

Se presentan y comparan tres aproximaciones al problema de la Subsecuencia Común Más Larga (LCS): una versión recursiva directa, la estrategia de divide y vencerás de Hirschberg y una solución mediante programación dinámica tabular. Se analiza su complejidad temporal y espacial, y se contrastan experimentalmente sobre cinco parejas de cadenas de ADN de longitud creciente hasta 1000 símbolos. Los resultados confirman el comportamiento exponencial de la versión recursiva, el uso lineal de memoria de Hirschberg y el equilibrio entre simplicidad y consumo de memoria de la versión tabular. El código fuente completo, el protocolo experimental y los resultados brutos se incluyen en los anexos de este mismo documento y en el repositorio público `https://github.com/carpediemservus/lcs-actividad1`.

Palabras clave: LCS, divide y vencerás, Hirschberg, programación dinámica, análisis de complejidad.

1. Introducción

Dadas dos cadenas X e Y de longitudes m y n, la *Subsecuencia Común Más Larga* (LCS) es una subsecuencia de longitud máxima que aparece en ambas cadenas conservando el orden relativo de los símbolos, aunque no necesariamente de forma contigua. El problema tiene aplicaciones directas en bioinformática (alineamiento de ADN y proteínas), en la comparación de versiones de ficheros (`diff`), en correctores ortográficos y en detección de plagio.

El objetivo de esta actividad es poner en práctica tres estrategias algorítmicas clásicas sobre el mismo problema, medir empíricamente sus diferencias de tiempo y memoria, y justificar las elecciones de diseño en función de sus respectivas complejidades. Las tres versiones implementadas devuelven la propia subsecuencia, no solo su longitud:

- **Versión A** — recursión directa, sin memorización.
- **Versión B** — divide y vencerás según el algoritmo de Hirschberg.
- **Versión C** — programación dinámica tabular.

El resto del documento se organiza como sigue. La Sección 2 describe el diseño de cada algoritmo, su análisis de complejidad y el protocolo experimental. La Sección 3 recoge las mediciones de tiempo de CPU y memoria máxima. La Sección 4 interpreta los resultados y la Sección 5 cierra con las conclusiones. Los anexos contienen el código fuente completo, los datos empleados y las instrucciones de reproducción local.

2. Metodología

2.1. Definición recurrente del problema

Sea $L(i, j)$ la longitud de la LCS de los prefijos $X[0..i)$ e $Y[0..j)$. Se cumple:

$$L(i,j) = \begin{cases} 0, & i = 0 \text{ o } j = 0, \\ L(i-1, j-1) + 1, & X[i-1] = Y[j-1], \\ \max\{L(i-1,j),\, L(i,j-1)\}, & \text{en otro caso.} \end{cases}$$

2.2. Versión A — recursión directa

Se traduce literalmente la recurrencia anterior a una función recursiva que devuelve la subsecuencia de cada subproblema: si los símbolos coinciden se añade el símbolo a la solución de $(i-1, j-1)$, y si no se elige la más larga de las dos ramas. No se almacena ningún resultado intermedio, por lo que cada subproblema se recalcula un número exponencial de veces. Su estado adicional es la pila de recursión y las cadenas parciales de cada nivel.

2.3. Versión B — divide y vencerás de Hirschberg

El algoritmo de Hirschberg [1] obtiene la LCS en tiempo $O(mn)$ y espacio $O(\min\{m, n\})$, evitando la tabla bidimensional. La idea es:

1. Partir X en dos mitades X_1 y X_2 de tamaños similares.
2. Calcular, mediante programación dinámica unidimensional, los vectores de longitudes $L_1(j) = L(|X_1|, j)$ y $L_2(j) = L(|X_2|, n-j)$.
3. Elegir el punto de corte k que maximiza $L_1(k) + L_2(k)$.
4. Resolver recursivamente $(X_1, Y[0..k))$ y $(X_2, Y[k..n))$ y concatenar las dos subsecuencias obtenidas. El caso base ($|X| = 1$) devuelve ese símbolo si aparece en Y.

La recursión alcanza profundidad $O(\log m)$. En el nivel k los subproblemas suman como mucho $mn/2^k$ comparaciones, de modo que el total está acotado por $mn(1 + \frac{1}{2} + \frac{1}{4} + \dots) \le 2mn$; de ahí el coste temporal $O(mn)$, con una constante aproximadamente doble que la de la DP tabular. La memoria auxiliar está dominada por los vectores de tamaño n, por lo que es $O(n)$ (o $O(\min\{m, n\})$ si se traspone la cadena más corta).

2.4. Versión C — programación dinámica tabular

Se rellena una matriz *dp* de tamaño $(m+1) \times (n+1)$ siguiendo el orden natural fila a fila. El valor final $dp[m][n]$ es la longitud de la LCS. El coste temporal es $O(mn)$ y el espacial es $O(mn)$ por la matriz completa. La subsecuencia se reconstruye recorriendo la tabla desde (m, n) hacia $(0, 0)$: en diagonal cuando los símbolos coinciden y, en otro caso, hacia la celda vecina con mayor valor. Este recorrido cuesta $O(m + n)$. Si solo interesara la *longitud* bastaría con dos filas ($O(n)$), pero la reconstrucción necesita la tabla completa.

2.5. Análisis de complejidad

La Tabla 1 resume las complejidades teóricas de las tres versiones. $T(n)$ denota el caso en que $m = n$; para el caso general basta sustituir n por $\max\{m, n\}$ en la cota superior.

Versión	Tiempo	Memoria	Observación
A · Recursiva directa	$O(2^{m+n})$	$O(m+n)$ pila	Inviable a partir de $n \approx 18$
B · Hirschberg	$\Theta(mn)$	$O(\min\{m,n\})$	Recursión con coste lineal por nivel
C · DP tabular	$\Theta(mn)$	$\Theta(mn)$	Matriz completa

Tabla 1: Complejidad temporal y espacial de las tres versiones.

2.6. Diseño experimental

Datos. Se generaron cadenas aleatorias sobre el alfabeto $\{A, C, G, T\}$ (ADN) con un generador propio, garantizando reproducibilidad mediante semilla fija $s = 42$. Se construyeron cinco parejas con longitudes $n \in \{50, 100, 250, 500, 1000\}$, todas con $m = n$. Para la Versión A, dado su coste exponencial, se emplearon longitudes adicionales $n \in \{8, 10, 12, 14, 16, 18, 20\}$.

Métricas. Para cada ejecución se registran:

- tiempo de CPU medido con `time.process_time()` (excluye espera de E/S);
- memoria pico medida con `tracemalloc` en un subproceso independiente, para aislar el consumo del propio algoritmo.

Protocolo. Cada punto experimental se repite cinco veces y se reporta la mediana. Se fija un *timeout* de 60 s por punto, que incluye las cinco repeticiones y la ejecución adicional de medida de memoria. El *script* `benchmark.py` (Anexo A.4) genera el CSV y las gráficas que alimentan la Sección 3.

Entorno. Intel Xeon a 2,10 GHz, Linux x86_64, CPython 3.13, sin otras cargas significativas durante la ejecución.

3. Resultados

Los valores de esta sección proceden de una ejecución de `make run` en el entorno descrito. El Anexo B recoge el CSV completo y el Anexo C las instrucciones para reproducirlos. Los tiempos absolutos variarán de un equipo a otro; las longitudes de LCS son idénticas en cualquier ejecución.

3.1. Versión A — crecimiento exponencial

La Tabla 2 muestra el comportamiento de la recursión directa. El tiempo se multiplica por un factor de entre 10 y 17 con cada incremento de dos unidades de n, es decir, algo más de 3^n en la práctica. Con $n = 16$ cada llamada tarda ya 1,3 s, y a partir de $n = 18$ el punto experimental supera el *timeout*.

n	Tiempo CPU (mediana)	Memoria pico	LCS
8	0,04 ms	0,6 KB	5
10	0,60 ms	0,8 KB	6
12	10 ms	0,9 KB	5
14	126 ms	1,0 KB	7
16	1,30 s	1,0 KB	8
18	*timeout*	—	—
20	*timeout*	—	—

Tabla 2: Versión A sobre cadenas de ADN de pequeña longitud.

3.2. Versiones B y C — comparación a gran escala

La Tabla 3 contrasta Hirschberg y la DP tabular en las cinco longitudes exigidas. Ambas resuelven el caso de 1000 símbolos en menos de 0,2 s, pero difieren sustancialmente en memoria.

	B · Hirschberg		**C · DP tabular**		
n	tiempo (ms)	memoria	tiempo (ms)	memoria	LCS
50	0,50	3,2 KB	0,22	21 KB	28
100	1,72	5,6 KB	0,85	83 KB	60
250	8,27	10 KB	4,08	507 KB	162
500	32,4	19 KB	21,3	2,1 MB	322
1000	159,4	66 KB	80,8	11,5 MB	644

Tabla 3: Comparación de las versiones B y C sobre cinco parejas de cadenas.

La Figura 1 representa gráficamente ambos indicadores en escala logarítmica, junto con la Versión A en la gráfica de tiempo. El tiempo de B y C crece de forma cuadrática con n: entre $n = 50$ y $n = 1000$ la pendiente log-log es 1,9 para Hirschberg y 2,0 para la DP tabular. La memoria sigue pendientes muy distintas: 2,1 para la DP tabular y 1,0 para Hirschberg, coherente con $\Theta(mn)$ y $O(\min\{m, n\})$ respectivamente.

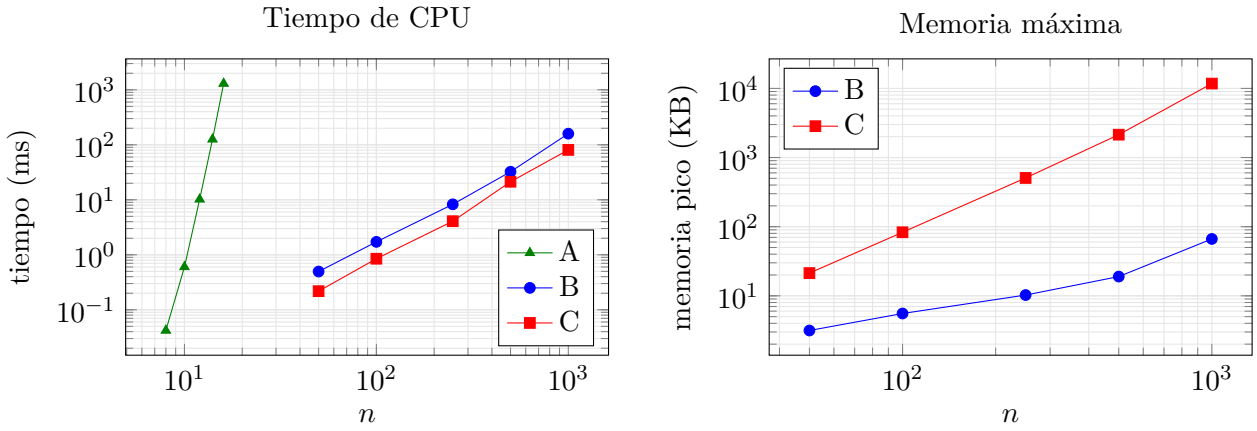


Figura 1: Tiempo de CPU y memoria máxima en función de n (escala log-log).

4. Discusión

Recursión directa: coste teórico confirmado. Los datos de la Tabla 2 verifican el comportamiento exponencial predicho por el análisis. El tiempo se multiplica por unas diez veces cada dos unidades de n, lo que refleja la estructura de árbol binario de la recurrencia. El crecimiento observado queda por debajo de la cota $2^{m+n} = 4^n$ porque cada coincidencia de símbolos genera una sola llamada en lugar de dos; con alfabeto de cuatro letras, aproximadamente una de cada cuatro comparaciones es una coincidencia. Para $n \leq 14$ la versión responde en menos de 0,2 s, con $n = 18$ ya no completa las seis ejecuciones en 60 s, y para $n = 1000$ sería inabordable.

La memoria pico medida apenas pasa de 0,6 a 1,0 KB, y corresponde a las cadenas parciales que se construyen en cada nivel. `tracemalloc` registra los objetos creados en el *heap*, pero no los marcos de la pila de recursión, que CPython 3.11+ gestiona en una zona propia. El consumo real crece linealmente con la profundidad de la pila, que nunca supera $m + n$ llamadas anidadas, de acuerdo con la cota $O(m + n)$.

Hirschberg frente a DP tabular. Ambas versiones comparten la cota temporal $\Theta(mn)$, pero Hirschberg es entre 1,5 y 2,3 veces más lenta según el tamaño. La causa es la que anticipa el análisis: el algoritmo recorre del orden de $2mn$ celdas frente a las mn de la DP tabular, y además paga las llamadas recursivas, las copias de subcadenas y la inversión de cadenas en cada nivel. La diferencia se vuelve decisiva en memoria: para $n = 1000$, la tabla completa ocupa 11,5 MB, mientras que Hirschberg necesita 66 KB, unas 180 veces menos. Las dos versiones obtienen siempre la misma longitud, aunque no necesariamente la misma subsecuencia: con $n = 1000$ devuelven cadenas distintas de 644 símbolos, ambas óptimas, porque el problema admite varias soluciones y cada algoritmo deshace los empates de forma diferente.

El salto de memoria de Hirschberg entre $n = 500$ (18 KB) y $n = 1000$ (66 KB) es mayor que el factor 2 esperado. Se explica por un detalle de CPython: los enteros entre -5 y 256 están precalculados y no ocupan memoria nueva, mientras que los mayores se crean como objetos de 28 bytes. Con $n = 1000$ los valores de los vectores de longitudes superan 256 y cada celda pasa a reservar un objeto propio. La pendiente asintótica sigue siendo lineal. El mismo efecto contribuye a que la memoria de la DP tabular crezca algo más deprisa que n^2 en el último tramo.

¿Cuándo elegir cada versión? La elección depende del recurso crítico:

- Si solo se necesita la *longitud*, basta una DP con dos filas, que es la más simple y rápida.
- Si se necesita la subsecuencia y mn cabe en memoria, la DP tabular con reconstrucción es la opción más directa.
- Si se necesita reconstruir la subsecuencia y las cadenas son grandes (genómica), Hirschberg es la opción profesional: mismo coste temporal, memoria lineal.
- La versión recursiva directa tiene valor didáctico, pero no de producción.

Limitaciones del experimento. Las mediciones se tomaron sobre un único equipo, con cadenas aleatorias de alfabeto uniforme. Los tiempos absolutos dependen fuertemente del lenguaje y de la implementación: en Python el intérprete domina, mientras que en C/C++ las diferencias entre B y C serían menores en tiempo. La estructura de los datos (por ejemplo, alfabetos muy pequeños o cadenas con muchos símbolos repetidos) también altera el factor constante, aunque no la complejidad asintótica.

5. Conclusiones

Se han diseñado, analizado y comparado empíricamente tres algoritmos para el problema de la LCS. Los experimentos confirman:

1. que la **recursión directa** incurre en un coste exponencial que la invalida a partir de $n \approx 18$ en el entorno de pruebas, tal y como predice el análisis;
2. que la **estrategia de Hirschberg** alcanza el mismo orden temporal que la DP tabular con un consumo de memoria dos órdenes de magnitud menor (66 KB frente a 11,5 MB con $n = 1000$), a cambio de duplicar aproximadamente el tiempo, y es la opción preferente cuando el espacio es un recurso crítico;
3. que la **DP tabular** es la solución más directa y la de menor coste constante, y resulta adecuada cuando la tabla cabe en memoria.

Como trabajo futuro se propone extender el estudio a cadenas con estructura real (por ejemplo, secuencias genómicas), añadir una versión memorizada (*top-down*) de la recursión, y repetir los experimentos en un lenguaje compilado para cuantificar el peso del intérprete.

Referencias

[1] D. S. Hirschberg, *A linear space algorithm for computing maximal common subsequences*, Communications of the ACM, 18(6):341–343, 1975.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.ª ed., MIT Press, 2009, cap. 15.

[3] D. Sankoff y J. B. Kruskal, *Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison*, Addison-Wesley, 1983.

A. Código fuente completo

Este anexo contiene la totalidad del código entregado. Los ficheros están organizados de modo que, copiados en una carpeta local con la estructura indicada en el Anexo C, el proyecto se ejecuta con un único comando (`make run`). Todo el contenido está incrustado en este documento y también está disponible en el repositorio `https://github.com/carpediemservus/lcs-actividad1`.

A.1. Estructura del proyecto

```
.
|-- Makefile
|-- README.md
|-- src/
|   |-- lcs.py          # implementaciones A, B, C
|   |-- benchmark.py    # experimentos + CSV + gráficas
|   |-- test_lcs.py     # pruebas de correctitud
|   `-- analyze.py      # tablas LaTeX a partir del CSV
`-- results/            # generado por `make run`
    |-- resultados.csv
    |-- tiempo.png
    |-- memoria.png
    `-- datos/          # parejas X, Y y LCS de cada n
```

Índice de listados

1. `src/lcs.py` . . . . . A
2. `src/test_lcs.py` C
3. `src/benchmark.py` . . . . . E
4. `src/analyze.py` I
5. `Makefile` . . . . . K
6. `results/datos/n50.txt` L
7. `results/resultados.csv` L
8. Salida de `make test` . . . . . M

A.2. `src/lcs.py` — implementaciones A, B y C

Listado 1: `src/lcs.py`

```python
"""Tres implementaciones de la Subsecuencia Comun Mas Larga (LCS).

Version A: recursion directa, sin memorizacion.
Version B: divide y venceras de Hirschberg (memoria lineal).
Version C: programacion dinamica tabular con reconstruccion.

Las tres devuelven la propia subsecuencia (una cadena); su longitud se
obtiene con len().
"""
import sys

sys.setrecursionlimit(10000)
```

```python


# ---------- Version A: recursiva directa ----------
def lcs_recursiva(a: str, b: str) -> str:
    def rec(i: int, j: int) -> str:
        if i == 0 or j == 0:
            return ""
        if a[i - 1] == b[j - 1]:
            return rec(i - 1, j - 1) + a[i - 1]
        x = rec(i - 1, j)
        y = rec(i, j - 1)
        return x if len(x) >= len(y) else y

    return rec(len(a), len(b))


# ---------- Version B: Hirschberg ----------
def _last_row(a: str, b: str) -> list:
    """Ultima fila de la DP (longitudes LCS(a, b[:j])) en O(|b|) memoria."""
    n = len(b)
    prev = [0] * (n + 1)
    for ca in a:
        curr = [0] * (n + 1)
        for j in range(1, n + 1):
            if ca == b[j - 1]:
                curr[j] = prev[j - 1] + 1
            else:
                curr[j] = prev[j] if prev[j] >= curr[j - 1] else curr[j - 1]
        prev = curr
    return prev


def lcs_hirschberg(a: str, b: str) -> str:
    if not a or not b:
        return ""
    if len(a) == 1:
        return a if a in b else ""

    mid = len(a) // 2
    l1 = _last_row(a[:mid], b)
    l2 = _last_row(a[mid:][::-1], b[::-1])[::-1]

    # punto de corte k que maximiza L1(k) + L2(k)
    k, best = 0, -1
    for j in range(len(b) + 1):
        s = l1[j] + l2[j]
        if s > best:
            best, k = s, j

    return lcs_hirschberg(a[:mid], b[:k]) + lcs_hirschberg(a[mid:], b[k:])
```

```python

# ---------- Version C: DP tabular ----------
def lcs_dp(a: str, b: str) -> str:
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        ai = a[i - 1]
        row, prev = dp[i], dp[i - 1]
        for j in range(1, n + 1):
            if ai == b[j - 1]:
                row[j] = prev[j - 1] + 1
            else:
                row[j] = prev[j] if prev[j] >= row[j - 1] else
                   row[j - 1]

    # reconstruccion: recorrido inverso desde dp[m][n]
    out = []
    i, j = m, n
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:
            out.append(a[i - 1])
            i -= 1
            j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return "".join(reversed(out))
```

A.3. src/test_lcs.py — pruebas de correctitud

Listado 2: src/test_lcs.py

```python
"""Pruebas de correctitud de las tres versiones.

Para cada caso se comprueba que la cadena devuelta:
  1. es subsecuencia de ambas entradas, y
  2. tiene la longitud optima, calculada con una DP de
     referencia.
"""
import random

from lcs import lcs_recursiva, lcs_hirschberg, lcs_dp


def _reference_len(a: str, b: str) -> int:
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
```

```
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]


def es_subsecuencia(s: str, t: str) -> bool:
    it = iter(t)
    return all(c in it for c in s)


def run(trials: int = 300) -> None:
    rng = random.Random(0)
    cases = [("", ""), ("A", ""), ("", "A"), ("A", "A"), ("AB", "BA"),
             ("ACGT", "ACGT"), ("AAAA", "CCCC"), ("ABCBDAB", "BDCABA")]
    for _ in range(trials):
        a = "".join(rng.choice("ACGT") for _ in range(rng.randint(0, 12)))
        b = "".join(rng.choice("ACGT") for _ in range(rng.randint(0, 12)))
        cases.append((a, b))
    # algunos casos mayores solo para B y C
    big = []
    for n in (100, 300):
        a = "".join(rng.choice("ACGT") for _ in range(n))
        b = "".join(rng.choice("ACGT") for _ in range(n))
        big.append((a, b))

    versiones = [("A", lcs_recursiva), ("B", lcs_hirschberg), ("C", lcs_dp)]
    total = 0
    for a, b in cases + big:
        expected = _reference_len(a, b)
        for name, fn in versiones:
            if name == "A" and (a, b) in big:
                continue
            got = fn(a, b)
            assert len(got) == expected, (
                f"Version {name}: {a!r} {b!r} -> longitud {len(got)} != {expected}"
            )
            assert es_subsecuencia(got, a) and es_subsecuencia(got, b), (
                f"Version {name}: {got!r} no es subsecuencia comun de {a!r} y {b!r}"
            )
            total += 1
    print(f"OK: {len(cases)} casos en las 3 versiones y {len(big)} casos "
          f"grandes en B y C ({total} comprobaciones).")


```

```python
if __name__ == "__main__":
    run()
```

A.4. `src/benchmark.py` — protocolo experimental

Listado 3: `src/benchmark.py`

```python
"""Benchmarks de las tres versiones de LCS.

Para cada (version, n) lanza un subproceso que:
  - genera las cadenas con semilla fija,
  - calcula la LCS (la subsecuencia) y registra su longitud,
  - mide el tiempo de CPU (mediana de REPEATS),
  - mide la memoria pico de Python con tracemalloc.
El proceso padre aplica timeout, escribe un CSV, dos graficas (log-log) y
guarda las parejas de cadenas y las LCS obtenidas en results/datos/.

Uso:
    python3 src/benchmark.py --csv results/resultados.csv
"""
import argparse
import csv
import json
import random
import statistics
import subprocess
import sys
import time
import tracemalloc
from pathlib import Path

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))

ALPHABET = "ACGT"
SEED = 42
REPEATS = 5
TIMEOUT = 60  # segundos

CONFIGS = {
    "A": [8, 10, 12, 14, 16, 18, 20],
    "B": [50, 100, 250, 500, 1000],
    "C": [50, 100, 250, 500, 1000],
}


# ---------------------- worker ----------------------
def _gen(n: int, seed: int) -> tuple:
    rng = random.Random(seed)
    return (
        "".join(rng.choice(ALPHABET) for _ in range(n)),
```

```
        "".join(rng.choice(ALPHABET) for _ in range(n)),
    )


def worker(version: str, n: int, seed: int) -> None:
    from lcs import lcs_recursiva, lcs_hirschberg, lcs_dp
    fn = {"A": lcs_recursiva, "B": lcs_hirschberg, "C":
       lcs_dp}[version]

    a, b = _gen(n, seed)

    # tiempo (mediana de REPEATS)
    times, result = [], None
    for _ in range(REPEATS):
        t0 = time.process_time()
        result = fn(a, b)
        times.append(time.process_time() - t0)

    # memoria pico (asignaciones Python durante la llamada)
    tracemalloc.start()
    fn(a, b)
    peak = tracemalloc.get_traced_memory()[1]
    tracemalloc.stop()

    print(json.dumps({
        "version": version,
        "n": n,
        "time_s": statistics.median(times),
        "mem_kb": peak / 1024.0,
        "lcs": len(result),
        "sub": result,
    }))


# ---------------------- driver ----------------------
def run_one(version: str, n: int) -> dict:
    cmd = [sys.executable, str(Path(__file__).resolve()),
           "--worker", version, str(n), str(SEED)]
    try:
        out = subprocess.run(cmd, capture_output=True, text=True,
                             timeout=TIMEOUT, check=True)
        return json.loads(out.stdout)
    except subprocess.TimeoutExpired:
        return {"version": version, "n": n, "time_s": None,
                "mem_kb": None, "lcs": None, "status": "timeout"}


def write_csv(rows, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["version", "n", "time_ms", "mem_kb", "lcs",
           "status"])
```

```
        for r in rows:
            t = "" if r.get("time_s") is None else
                f"{r['time_s']*1000:.3f}"
            m = "" if r.get("mem_kb") is None else
                f"{r['mem_kb']:.2f}"
            w.writerow([r["version"], r["n"], t, m,
                        r.get("lcs", ""), r.get("status", "ok")])


def write_data(rows, outdir: Path) -> None:
    """Guarda cada pareja (X, Y) y la LCS obtenida por la
       version C."""
    d = outdir / "datos"
    d.mkdir(parents=True, exist_ok=True)
    for r in rows:
        if r["version"] != "C" or r.get("sub") is None:
            continue
        x, y = _gen(r["n"], SEED)
        (d / f"n{r['n']}.txt").write_text(
            f"X={x}\nY={y}\nLCS={r['sub']}\n", encoding="utf-8")
    print(f"[benchmark] datos -> {d}")


def write_plots(rows, outdir: Path) -> None:
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        print("[benchmark] matplotlib no instalado: se omiten
           las graficas.")
        return

    outdir.mkdir(parents=True, exist_ok=True)
    colors = {"A": "tab:green", "B": "tab:blue", "C": "tab:red"}
    marks = {"A": "^", "B": "o", "C": "s"}

    for field, fname, ylabel, title in [
        ("time_s", "tiempo.png", "tiempo CPU (s)", "Tiempo de
           CPU"),
        ("mem_kb", "memoria.png", "memoria pico (KB)", "Memoria
           maxima"),
    ]:
        fig, ax = plt.subplots(figsize=(5.5, 4))
        for v in ("A", "B", "C"):
            pts = [(r["n"], r[field]) for r in rows
                   if r["version"] == v and r.get(field) is not
                      None]
            if not pts:
                continue
            xs, ys = zip(*sorted(pts))
            ax.plot(xs, ys, marker=marks[v], color=colors[v],
                    label=f"Version {v}")
```

```
        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.set_xlabel("n")
        ax.set_ylabel(ylabel)
        ax.set_title(title)
        ax.grid(True, which="both", ls=":", alpha=0.4)
        ax.legend()
        fig.tight_layout()
        fig.savefig(outdir / fname, dpi=150)
        plt.close(fig)
        print(f"[benchmark] grafica -> {outdir / fname}")


# ----------------------- main -----------------------
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--worker", nargs=3, metavar=("VERSION",
       "N", "SEED"))
    ap.add_argument("--csv", default="results/resultados.csv")
    args = ap.parse_args()

    if args.worker:
        v, n, s = args.worker
        worker(v, int(n), int(s))
        return

    rows = []
    for version, sizes in CONFIGS.items():
        for n in sizes:
            print(f"[benchmark] version {version}, n={n} ...",
               flush=True)
            r = run_one(version, n)
            rows.append(r)
            if r.get("status") == "timeout":
                print("   -> timeout")
            else:
                print(f"   -> t={r['time_s']*1000:8.2f} ms   "
                      f"mem={r['mem_kb']:8.1f} KB
                         LCS={r['lcs']}")

    csv_path = Path(args.csv)
    write_csv(rows, csv_path)
    print(f"[benchmark] CSV -> {csv_path}")
    write_data(rows, csv_path.parent)
    write_plots(rows, csv_path.parent)


if __name__ == "__main__":
    main()
```

A.5. `src/analyze.py` — generación de tablas LaTeX

Listado 4: src/analyze.py

```python
"""Convierte results/resultados.csv en las Tablas 2 y 3 del
   informe (LaTeX).

Uso:
    python3 src/analyze.py
    python3 src/analyze.py --csv otro.csv
    python3 src/analyze.py --out tablas.tex
"""
import argparse
import csv
from pathlib import Path


def fmt_time(ms):
    """Tiempo en ms, con 2 decimales y separador decimal LaTeX
       (coma)."""
    if ms is None or ms == "":
        return "---"
    v = float(ms)
    if v >= 1000:
        s = f"{v/1000:.2f}".rstrip("0").rstrip(".")
        s = s.replace(".", "{,}")
        return f"{s}\\,s"
    s = f"{v:.2f}".rstrip("0").rstrip(".")
    return s.replace(".", "{,}")


def fmt_mem(kb):
    """Memoria: KB si <1024, MB en otro caso (con 2
       decimales)."""
    if kb is None or kb == "":
        return "---"
    v = float(kb)
    if v >= 1024:
        s = f"{v/1024:.1f}".rstrip("0").rstrip(".")
        return s.replace(".", "{,}") + "\\,MB"
    s = f"{v:.1f}" if v < 10 else f"{v:.0f}"
    return s.replace(".", "{,}") + "\\,KB"


def fmt_lcs(v):
    return "---" if v in (None, "") else str(v)


def load(csv_path):
    rows = []
    with open(csv_path, newline="") as f:
        for r in csv.DictReader(f):
            rows.append({
                "version": r["version"],
                "n": int(r["n"]),
```

```
                "time_ms": None if r["time_ms"] == "" else
                   float(r["time_ms"]),
                "mem_kb":  None if r["mem_kb"] == ""  else
                   float(r["mem_kb"]),
                "lcs":     None if r["lcs"] == ""     else
                   r["lcs"],
                "status":  r.get("status", "ok"),
            })
    return rows


def by_version(rows, v):
    return sorted([r for r in rows if r["version"] == v],
       key=lambda r: r["n"])


def table_A(rows):
    out = [
        r"\begin{table}[h]", r"\centering",
        r"\begin{tabular}{@{}rrrr@{}}", r"\toprule",
        r"$n$ & Tiempo CPU (mediana) & Memoria pico & LCS \\",
        r"\midrule",
    ]
    for r in by_version(rows, "A"):
        t = "\\textit{timeout}" if r["status"] == "timeout" else
           fmt_time(r["time_ms"])
        m = fmt_mem(r["mem_kb"])
        l = fmt_lcs(r["lcs"])
        out.append(f"{r['n']:<4} & {t:<20} & {m:<10} & {l} \\\\")
    out += [r"\bottomrule", r"\end{tabular}",
            r"\caption{Version A.}\label{tab:A}", r"\end{table}"]
    return "\n".join(out)


def table_BC(rows):
    b = {r["n"]: r for r in by_version(rows, "B")}
    c = {r["n"]: r for r in by_version(rows, "C")}
    ns = sorted(set(b) | set(c))
    out = [
        r"\begin{table}[h]", r"\centering",
        r"\begin{tabular}{@{}rrrrrr@{}}", r"\toprule",
        r"& \multicolumn{2}{c}{\textbf{B - Hirschberg}} "
        r"& \multicolumn{2}{c}{\textbf{C - DP tabular}} & \\",
        r"\cmidrule(lr){2-3}\cmidrule(lr){4-5}",
        r"$n$ & tiempo (ms) & memoria & tiempo (ms) & memoria &
           LCS\\",
        r"\midrule",
    ]
    for n in ns:
        rb, rc = b.get(n, {}), c.get(n, {})
        out.append(
            f"{n:<5} & {fmt_time(rb.get('time_ms')):<10} & "
            f"{fmt_mem(rb.get('mem_kb')):<9} & "
```

```
            f"{fmt_time(rc.get('time_ms')):<10} & "
            f"{fmt_mem(rc.get('mem_kb')):<9} & "
            f"{fmt_lcs(rb.get('lcs') or rc.get('lcs'))} \\\\"
        )
    out += [r"\bottomrule", r"\end{tabular}",
            r"\caption{Versiones B y C.}\label{tab:BC}",
               r"\end{table}"]
    return "\n".join(out)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--csv", default="results/resultados.csv")
    ap.add_argument("--out", default=None)
    args = ap.parse_args()

    rows = load(Path(args.csv))
    text = ("% === Tabla 2 ===\n" + table_A(rows)
            + "\n\n% === Tabla 3 ===\n" + table_BC(rows) + "\n")
    if args.out:
        Path(args.out).write_text(text, encoding="utf-8")
        print(f"Tablas escritas en {args.out}")
    else:
        print(text)


if __name__ == "__main__":
    main()
```

A.6. Makefile

Listado 5: Makefile

```
PYTHON ?= python3
CSV    := results/resultados.csv

.PHONY: all run test analyze clean help

all: run

run:
        @mkdir -p results
        $(PYTHON) src/benchmark.py --csv $(CSV)

test:
        $(PYTHON) src/test_lcs.py

analyze: run
        $(PYTHON) src/analyze.py --csv $(CSV) --out results/
           tablas.tex
        @echo "Tablas␣generadas␣en␣results/tablas.tex"

clean:
```

```
20          rm -rf results
21          find . -type d -name __pycache__ -exec rm -rf {} + 2>/dev
               /null || true
22
23  help:
24          @echo "make␣run␣␣␣␣␣-␣ejecuta␣benchmarks␣(CSV␣+␣graficas)
               "
25          @echo "make␣test␣␣␣␣-␣pruebas␣de␣correctitud"
26          @echo "make␣analyze␣-␣vuelca␣Tablas␣2␣y␣3␣en␣LaTeX"
27          @echo "make␣clean␣␣␣␣-␣limpia␣artefactos"
```

B. Datos y resultados brutos

B.1. Generación de los datos

Todas las cadenas empleadas en el informe se generan de forma determinista con el siguiente procedimiento (función `_gen` del Anexo A.4):

- Alfabeto: $\{A, C, G, T\}$.
- Semilla: $s = 42$.
- Para cada n, se construyen dos cadenas independientes $X, Y \in \{A, C, G, T\}^n$ usando un generador pseudoaleatorio `random.Random(42)`.

Al fijar la semilla, cualquier ejecución de `make run` reproduce exactamente las mismas cadenas y, por tanto, los mismos valores de LCS. Las cinco parejas completas, junto con la LCS obtenida por la Versión C, se guardan en `results/datos/n<n>.txt` y están publicadas en el repositorio. Como muestra, la pareja de $n = 50$ es:

Listado 6: `results/datos/n50.txt`

```
X=AAGCCCAATAAACCACTCTGACTGGCCGAATAGGGATATAGGCAACGACA
Y=TGTGCGGCGACCCTTGCGACAGTGACGCTTTCGCCGTTGCCTAAACCTAT
LCS=GCCACCCTCGACTGGCCGGTTGCAACCA
```

La Tabla 4 recoge la longitud de la LCS obtenida para cada par. Las versiones B y C coinciden en todos los casos, y el cociente LCS/n tiende a 0,64–0,65, el valor esperado para cadenas aleatorias sobre un alfabeto de cuatro símbolos.

n	50	100	250	500	1000
LCS	28	60	162	322	644

Tabla 4: Longitud de la LCS para cada pareja de cadenas (semilla $s = 42$).

B.2. Volcado del CSV de resultados

El Anexo A.4 genera el siguiente fichero `results/resultados.csv`. Se incluye aquí como evidencia de los tiempos y memoria reportados en la Sección 3.

Listado 7: `results/resultados.csv`

```
version,n,time_ms,mem_kb,lcs,status
A,8,0.042,0.64,5,ok
```

```
A,10,0.602,0.82,6,ok
A,12,10.203,0.90,5,ok
A,14,125.731,0.99,7,ok
A,16,1298.861,1.04,8,ok
A,18,,,,timeout
A,20,,,,timeout
B,50,0.496,3.15,28,ok
B,100,1.718,5.55,60,ok
B,250,8.272,10.26,162,ok
B,500,32.370,18.98,322,ok
B,1000,159.369,66.46,644,ok
C,50,0.218,21.31,28,ok
C,100,0.848,82.97,60,ok
C,250,4.080,506.66,162,ok
C,500,21.289,2140.45,322,ok
C,1000,80.841,11736.25,644,ok
```

B.3. Pruebas de correctitud

La ejecución de `make test` (Anexo A.3) comprueba, para cada caso, que la cadena devuelta es subsecuencia de las dos entradas y que su longitud coincide con la óptima calculada por una DP de referencia. Se usan 308 casos con las tres versiones (ocho casos borde y 300 pares aleatorios de longitud entre 0 y 12) y dos pares de 100 y 300 símbolos solo con B y C, en total 928 comprobaciones. Todas finalizan con éxito:

Listado 8: Salida de `make test`

```
OK: 308 casos en las 3 versiones y 2 casos grandes en B y C (928
   comprobaciones).
```

C. Instrucciones de reproducción local

Todo el código y los datos necesarios están contenidos en este documento y en el repositorio público `https://github.com/carpediemservus/lcs-actividad1`. Para reproducir los resultados en un equipo local:

1. Clone el repositorio: `git clone https://github.com/carpediemservus/lcs-actividad1`. Como alternativa, cree una carpeta de trabajo y copie en ella los ficheros del Anexo A respetando la estructura indicada (carpeta `src/` y `Makefile` en la raíz).
2. Verifique que dispone de Python 3.9 o superior: `python3 -version`.
3. (Opcional) Instale `matplotlib` si desea regenerar las gráficas: `pip install matplotlib`.
4. Ejecute:

```
make test      # valida las 3 versiones contra una DP de referencia
make run       # ejecuta los benchmarks, genera CSV y graficas
make analyze   # vuelca las Tablas 2 y 3 en LaTeX
```

Los artefactos quedan en `results/`:

- `resultados.csv` — datos brutos de todas las mediciones;

- `tiempo.png`, `memoria.png` — gráficas log-log;
- `tablas.tex` — Tablas 2 y 3 listas para pegar en el informe.

El tiempo total de ejecución de `make run` es de aproximadamente 2–3 minutos, de los que 120 s corresponden a los dos *timeouts* de la Versión A ($n = 18$ y $n = 20$). El compilado del informe no requiere ningún paso adicional más allá de `pdflatex informe.tex` dos veces.